

ICS 435
Machine Learning
Spring 2026
Assignment 4

Part I. Convolutional Neural Networks (CNN) for FashionMNIST

Section 1. Task Description

The task studied in this assignment is image classification on the FashionMNIST dataset. The goal is to train a Convolutional Neural Network (CNN) to recognize grayscale clothing images and assign them to one of 10 categories, such as T-shirt, trouser, dress, sneaker, and bag. This assignment focuses on the full machine learning workflow, including data preprocessing, model construction, training, evaluation, and performance improvement through experimentation.

Section 2. Model Description

In this assignment, we compared three different CNN-based models:

Baseline CNN

The baseline model consists of:

- Two convolutional layers
- Max-pooling layers
- Two fully connected layers

This model serves as the reference architecture for performance comparison.

CNN with Batch Normalization and Dropout

This model extends the baseline by:

- Adding batch normalization after convolutional layers
- Adding dropout (rate = 0.5) before the final classification layer

These changes improve training stability and reduce overfitting.

Deeper CNN with Data Augmentation

This model further improves performance by:

- Adding additional convolutional layers
- Using batch normalization and dropout (rate = 0.4)
- Applying data augmentation (random horizontal flips and small rotations)

The goal is to improve generalization and classification accuracy.

Section 3. Experiment Settings

3.1 Dataset Description

The FashionMNIST dataset contains:

- 70,000 grayscale images
- Image size: 28×28 pixels
- 10 classes

Dataset split:

- 60,000 training images
- 10,000 testing images

In this assignment, training set split into 80% training and 20% validation

3.2 Experimental Setup

- Batch size: 64
- Epochs: 5
- Optimizer: Adam
- Learning rate: 0.001
- Loss function: Cross-Entropy Loss
- Random seed: 42

Model Variations

- Baseline CNN: 2 convolutional layers (32, 64 filters)
- BatchNorm + Dropout CNN: + batch normalization and dropout (0.5)
- Deeper CNN: more layers + dropout (0.4) + data augmentation

3.3 Evaluation Metrics

The following metrics were used:

- Accuracy: Overall proportion of correct predictions
- Precision: Correctness of predicted labels
- Recall: Ability to identify all true examples
- F1-Score: Harmonic mean of precision and recall

Weighted averages were used across all 10 classes.

3.4 Source Code

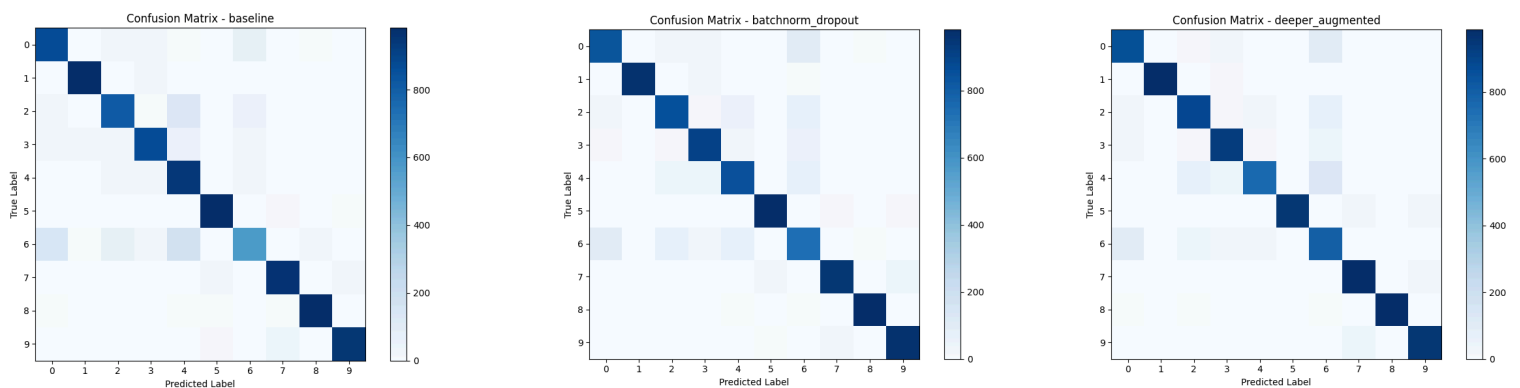
GitHub Repository:

<https://github.com/jkugiyama/ICS-435-Assignment-4>

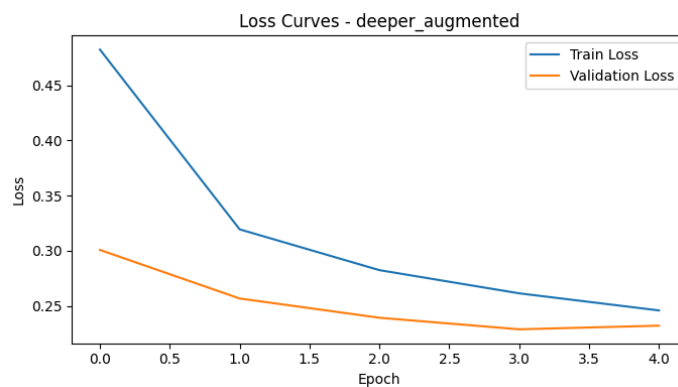
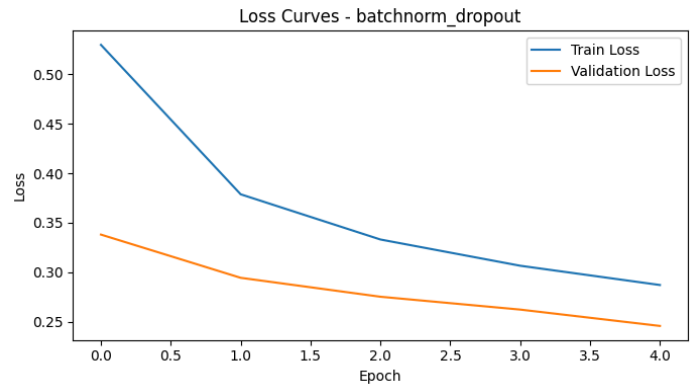
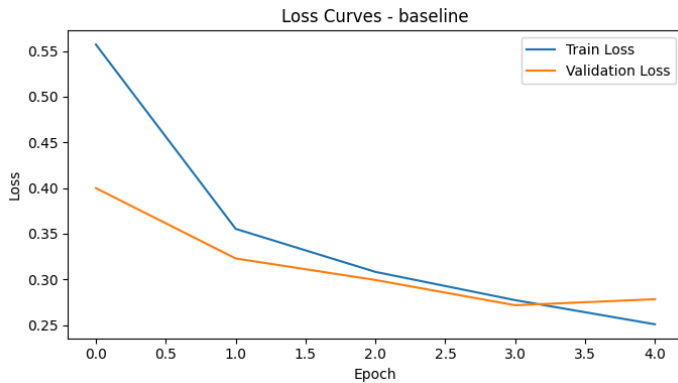
3.5 Model Performance

Model	Accuracy	Precision	Recall	F1-Score
Baseline CNN	0.8900	0.8923	0.8900	0.8880
BatchNorm + Dropout CNN	0.9036	0.9048	0.9036	0.9041
Deeper CNN + Augmentation	0.9094	0.9133	0.9094	0.9101

Confusion Matrices:



Loss Curves:



3.6 Ablation Study

The ablation study shows clear performance improvements:

- Adding batch normalization and dropout improved accuracy from 0.8900 $\rightarrow$ 0.9036
- Adding deeper architecture and data augmentation improved accuracy further to 0.9094

These results indicate that:

- Regularization improves generalization
- Deeper networks extract more meaningful features

Section 4. Conclusion

This assignment demonstrates that CNNs are highly effective for FashionMNIST classification.

- The baseline CNN provided a strong starting point
- The BatchNorm + Dropout model improved stability and reduced overfitting
- The Deeper CNN with augmentation achieved the best overall performance

Overall, combining deeper architectures with regularization and data augmentation leads to the highest classification accuracy and robustness.

Part II. Fine-Tuning GPT-2 for Joke Generation

Section 1. Task Description

The task in this assignment is to fine-tune a pre-trained GPT-2 language model to generate jokes. The model is trained on a dataset of 1,622 short jokes and is expected to produce a complete joke when given only the first three words as input. The goal is to evaluate whether the model can learn the structure of jokes (such as setup and punchline) and generalize this knowledge to both familiar prompts (seen in training) and unseen prompts (random word combinations).

Section 2. Model Description

This assignment uses the GPT-2 model, a transformer-based language model developed by OpenAI. GPT-2 is designed for text generation and is pre-trained on a large corpus of internet text using unsupervised learning. Some key characteristics of GPT-2 is that it uses a decoder-only transformer architecture. It predicts the next word in a sequence based on previous context and can generate coherent and contextually relevant text.

For this assignment, the base version of GPT-2 (117 million parameters) is fine-tuned on a joke dataset so that it learns humor-specific patterns, such as:

- Question-and-answer joke formats
- Timing and structure of punchlines
- Common comedic phrasing

Section 3. Experiment Settings

3.1 Dataset Description

The dataset consists of 1,622 short jokes, stored in CSV format. Each entry contains a single joke, typically following a short setup and punchline structure. No explicit train/test split is used; instead, evaluation is performed qualitatively using selected prompts.

3.2 Experimental Setup

The model is fine-tuned using the following configuration:

Setting	Value
Base model	GPT-2
Max sequence length	64 tokens
Batch size	8
Epochs	3
Optimizer	AdamW
Padding strategy	Dynamic padding
Tokenizer padding	EOS token used as pad
Grouping	Enabled (group_by_length=True)

During generation:

- Temperature = 0.9 (to increase diversity)
- Top-k sampling = 50 (to limit randomness)

Two evaluation sets are used:

1. Prompts from the dataset (first three words of real jokes)
2. Random prompts not seen during training

3.3 Evaluation Metrics

The evaluation in this assignment is qualitative rather than quantitative.

The following criteria are used to assess model performance:

- Joke structure: Does the output follow a recognizable setup and punchline?
- Coherence: Is the generated text logically consistent?
- Creativity: Does the model generate novel content rather than copying?
- Completion quality: Does the joke end properly without repetition?

No numerical metrics (accuracy or F1 score) are used due to the subjective nature of humor.

3.4 Source Code

GitHub Repository:

<https://github.com/jkugiyama/ICS-435-Assignment-4>

3.5 Model Performance

```
ID                                     Joke
0 1 What did the bartender say to the jumper cable...
1 2 Don't you hate jokes about German sausage? The...
2 3 Two artists had an art contest... It ended in ...
3 4 Why did the chicken cross the playground? To g...
4 5 What gun do you use to hunt a moose? A moosecut!

['What did the bartender say to the jumper cables? You better not try to start anything.', "Don't you hate jokes about German sausage? They're the wurst!", 'Two artists had an art contest... It ended in a draw', 'Why did the chicken cross the playground? To get to the other slide.', 'What gun do you use to hunt a moose? A moosecut!']
Map: 100% |██████████| 1622/1622 [00:00<00:00, 7976.21 examples/s]
    0%/ ██████████ | 0/609 [00:00<?, ?it/s] 'loss_type=None'

was set in the config but it is unrecognized. Using the default loss: `ForCausalLMLoss`.
{'loss': 3.4087, 'grad_norm': 15.6320@1876831055, 'learning_rate': 9.031198686371101e-06, 'epoch': 2.46}
{'train_runtime': 83.0021, 'train_samples_per_second': 58.625, 'train_steps_per_second': 7.337, 'train_loss': 3.3209579292385056, 'epoch': 3.0}
100% |██████████| 609/609 [01:23<00:00, 7.34it/s]
```

Performance on Dataset-Based Prompts

When prompts match training patterns, the model frequently produces recognizable joke formats, especially Q&A jokes. Examples like “What did the” and “Why did the” show strong structure.

However, outputs often repeat phrases, punchlines are sometimes weak, and the model struggles to end cleanly. Overall, about 4 out of 5 prompts produce joke-like outputs.

Performance on Random Prompts

When prompts are not seen during training, the model often defaults to general text generation (stories, facts, opinions), joke structure is mostly absent, and humor appears occasionally but is usually unintentional. Only 1–2 out of 5 prompts show noticeable joke-like behavior.

```

=== EVALUATION SET 1: Dataset-based prompts ===

Prompt : 'What did the'
Output : What did the frog say to the snail? "What's next?" The snail replied, "I have a rash!" ...and then the frog said, "Well, that's all it took ... It'll be a while, but you get used to it." "Why don't frogs use paper and pencil?"

Prompt : 'Two artists had'
Output : Two artists had a baby in the bathroom of a bar... So he started using his bathroom mirrors. And he's since stopped using his mirrors. *grin * *grin* *grin* *grin* *grin* So here's a link to the original. **https://www.

Prompt : 'Why did the'
Output : Why did the elephant go to the dentist? Because he was very sick. It was said that he had a bad bite. *Why did the elephant go to the dentist? Because he was very sick. It was said that he had a bad bite. I am sure he's not dead, but there was no

Prompt : 'What did one'
Output : What did one fish say to the other fish? Fish, you got me there. The one fish was too slow to catch my fish. So it's left up to me to get up to speed. *I* said to my other fish: Fish, you got me there. *I* said to my

Prompt : 'How many hipsters'
Output : How many hipsters does it take to change one light bulb? Two light bulbs. So, you might as well say there are eight light bulbs in here. But I'd have to say it's pretty common. It's not illegal, you could say, but it's a little high to take. I'll tell

=== EVALUATION SET 2: Random prompts (not from dataset) ===

Prompt : 'Never trust a'
Output : Never trust a cow with a toe! ...who could have guessed that? (Credit to the cow) ***Totally not a cow*** ***Not a cow at all. ***Beware of cow poo*** ***Slipper** (Credit to a cow) ***Beware of cow poo*** (

Prompt : 'Scientists recently discovered'
Output : Scientists recently discovered that there are four types of fish. Blue, brown, and black. They all have a common origin. They are called bluefish. -B.I.P.S.: What do you call a bluefish with no fins? A bluefish with a fin. -B.I.

Prompt : 'My dog always'
Output : My dog always gets lost on the road.. ..it is so sad. We lost his favorite thing! Dog-paws. He was so sad I couldn't take him on his dog walk. He had to go to the vet. He has a special breed of dog... but he was born with *p

Prompt : 'The president decided'
Output : The president decided to make his country famous by giving a green light to Russian oil. I think it was a great idea. But it did little or nothing to affect the mood in Russia. And I can't help but wonder if it's because I'm a lot more Russia-lovin' than the general.

Prompt : 'Once upon a'
Output : Once upon a time, the universe was flat. From a distance, it felt like a flat balloon. But when you put it on a high, it felt like it was ballooning. (I'm trying to make this up as best I can.) It turned out that the atoms had no air. And,

```

Key Observations

The model learns patterns, not exact jokes, which makes sense as most LLM are pattern based rather than sentiment based. It's difficult for machines to reproduce human emotion. Fine-tuning influence is limited by dataset size. Since the dataset that was used isn't the largest, the model isn't able to perform well on unseen phrases or prompts. Pre-training strongly affects outputs for unfamiliar prompts, giving the model false confidence in producing a quality output. At first, I had some trouble with the training segment, as it was taking a long time (~40 minutes), which I thought was odd since the dataset wasn't the largest.

Section 4. Conclusion

This assignment demonstrates that fine-tuning GPT-2 on a small joke dataset enables the model to learn basic joke structures, particularly for prompts similar to the training data. However, the model struggles to generalize to unseen prompts and often produces incoherent or non-humorous outputs. This highlights the limitations of small datasets and the strong influence of pre-trained language distributions.

Future improvements could include using a larger and more diverse joke dataset, training for more epochs, and applying better evaluation methods, including human feedback. Overall, the results show that while GPT-2 can learn structural patterns of humor, generating consistently high-quality jokes remains a challenging task.